

RootCause macOS Inspector

Documentación del sistema — RootCause macOS Inspector

Documentacion del sistema

Version analizada: **0.1.0**

Commit analizado: **2282a33**

Fecha de generacion: **2026-08-27**

Generado a partir de README.md — los Markdown son la fuente unica.
No editar este PDF: editar el Markdown y regenerar.

Contenido de este documento

1. Identificación
2. Descripción breve
3. Propósito de esta documentación
4. Público destinatario por documento
5. Tabla de contenidos
6. Cómo generar los PDF
7. Documentación añadida en el propio código
8. Relación con la documentación previa del repositorio
9. Convenciones utilizadas
10. Elementos pendientes de validar

1. Identificación

Campo	Valor
Sistema	RootCause macOS Inspector
Nombre del paquete	<code>rootcause-macos-inspector</code> (binario: <code>rootcause</code>)
Versión analizada	<code>0.1.0</code> (<code>Cargo.toml</code> , campo <code>version</code>)
Commit analizado	<code>2282a3365fcf90f88df8c4b0b5895680570004ac</code> (rama <code>main</code>)
Fecha del análisis	2026-08-27
Lenguaje principal	Rust (edición 2021, <code>rust-version = 1.82</code>)
Plataforma objetivo	macOS 13 o superior (Apple Silicon e Intel)
Licencia	Apache License 2.0
Repositorio	https://github.com/vladimiracunadev-create/rootcause-macos-inspector

2. Descripción breve

RootCause macOS Inspector es un **sensor forense de ciberseguridad y diagnóstico para macOS**, escrito en Rust. Observa siete superficies del equipo —persistencia de `launchd`, procesos, controles de seguridad nativos, definiciones antimalware de Apple, permisos de privacidad TCC, red y almacenamiento—, compara cada una contra una baseline de «estado bueno conocido», correlaciona las señales en incidentes y explica la causa probable con evidencia.

No es un antivirus ni un EDR: no elimina malware ni bloquea por firma. Entrega **indicios con evidencia**, no veredictos. Todo el análisis es local; la única salida a red posible es un adaptador de IA opcional apagado por defecto.

3. Propósito de esta documentación

Permitir que:

1. Un desarrollador nuevo se incorpore al proyecto sin depender de conocimiento tácito.
2. Una persona no técnica entienda qué hace el sistema y para qué sirve.
3. Un desarrollador experimentado consulte detalles de tipos, funciones y flujos.
4. Un auditor revise arquitectura, persistencia, dependencias, seguridad y deuda técnica.
5. Otro agente de IA use estos documentos como contexto verificable del repositorio.

4. Público destinatario por documento

Documento	Usuario final	Dev nuevo	Dev senior	Auditor	Dirección
01 · Descripción general	■	■	■	■	■

02 · Instalación y ejecución	■	■	■	■	■
03 · Arquitectura	■	■	■	■	■
04 · Mapa del código	■	■	■	■	■
05 · Referencia técnica	■	■	■	■	■
06 · Explicación profunda	■	■	■	■	■
07 · Base de datos	■	■	■	■	■
08 · Flujo de datos	■	■	■	■	■
09 · APIs e integraciones	■	■	■	■	■
10 · Configuración	■	■	■	■	■
11 · Seguridad	■	■	■	■	■
12 · Pruebas y calidad	■	■	■	■	■
13 · Despliegue y operación	■	■	■	■	■
14 · Solución de problemas	■	■	■	■	■
15 · Riesgos y deuda técnica	■	■	■	■	■
16 · Glosario	■	■	■	■	■
17 · Resumen ejecutivo	■	■	■	■	■
18 · Guía para nuevo desarrollador	■	■	■	■	■
19 · Matriz de trazabilidad	■	■	■	■	■

■ destinatario principal · ■ lectura opcional

5. Tabla de contenidos

#	Documento	Contenido	Estado
—	README	Portada, índice y convenciones	■ Completo
01	Descripción general del sistema	Qué es, qué resuelve, casos de uso, explicación no técnica	■ Completo

02	Instalación y ejecución	Requisitos, compilación, ejecución, pruebas, errores frecuentes	■ Completo
03	Arquitectura	Estilo, capas, patrones, diagramas Mermaid	■ Completo
04	Mapa completo del código	Inventario jerárquico de directorios, módulos y funciones	■ Completo
05	Referencia técnica	Catálogo de funciones, tipos, constantes, comandos y errores	■ Completo
06	Explicación profunda del código	Flujo interno módulo a módulo, decisiones y casos límite	■ Completo
07	Base de datos	Esquema SQLite, diccionario de datos, consultas, ERD	■ Completo
08	Flujo de datos	Origen, validación, transformación, almacenamiento, consumo	■ Completo
09	APIs e integraciones	CLI, JSON, utilidades nativas de macOS, IA opcional	■ Completo
10	Configuración	<code>rootcause-config.json</code> campo por campo, variables de entorno	■ Completo
11	Seguridad	Superficie de ataque, controles presentes y ausentes	■ Completo
12	Pruebas y calidad	112 tests, cobertura observable, huecos priorizados	■ Completo
13	Despliegue y operación	CI/CD, artefactos, release, logs, respaldo y rollback	■ Completo
14	Solución de problemas	Síntoma → causa → diagnóstico → solución	■ Completo
15	Riesgos y deuda técnica	Hallazgos clasificados por severidad e impacto	■ Completo
16	Glosario	Términos técnicos y de dominio en lenguaje claro	■ Completo
17	Resumen ejecutivo	Presentación del sistema para decisión	■ Completo
18	Guía para un nuevo desarrollador	Itinerario de incorporación y primeras tareas	■ Completo
19	Matriz de trazabilidad	Funcionalidad → módulo → función → persistencia → prueba	■ Completo

Recursos adicionales

- `assets/` — recursos gráficos de esta documentación. Actualmente vacío: los diagramas se entregan como código Mermaid dentro de los propios documentos, y la identidad visual del producto vive en `assets/rootcause-icon.svg`.
- `pdf/` — versión PDF de cada documento, generada por script.

6. Cómo generar los PDF

Los Markdown de esta carpeta son la **fente única**. Los PDF se generan a partir de ellos:

```
python3 -m pip install markdown xhtml2pdf
python3 scripts/build-docs-pdf.py
```

Detalle de requisitos, opciones y limitaciones conocidas en [13 · Despliegue y operación](#).

7. Documentación añadida en el propio código

Este análisis también completó la documentación **dentro del código fuente**. Estado antes y después, medido sobre los 23 archivos `.rs` de `src/`:

Métrica	Antes	Después
Archivos con documentación de módulo (<code>//!</code>)	23 / 23	23 / 23
Elementos públicos con documentación (<code>///</code>)	180 / 232 (78 %)	232 / 232 (100 %)
Líneas de código fuente	12 885	12 956

Se añadieron **52 bloques `///`** (71 líneas) en elementos públicos que no tenían documentación: los diez structs de configuración, los quince `pub mod` del módulo de servicios, los accesoros del motor de inspección, cuatro métodos `label()` de los modelos y el estado de la aplicación gráfica.

No se modificó ni una línea de código existente. El cambio es puramente aditivo: `git diff -- src/` muestra 71 inserciones, todas líneas `///`, y cero eliminaciones. `cargo fmt --all -- --check` devuelve 0 y `cargo test --all-features` pasa los 112 tests.

Las cifras de líneas de este conjunto de documentos reflejan el estado **posterior** a esa adición.

8. Relación con la documentación previa del repositorio

Este conjunto **no reemplaza** la documentación que ya existía en `docs/`; la complementa y la referencia. La documentación previa está orientada a producto, marca, roadmap y operación; esta está orientada a comprensión del sistema y auditoría.

Ya existía	Este conjunto aporta
<code>docs/PLAN_MAESTRO.md</code> — tesis y principios del producto	Estado verificado del código en el commit analizado
<code>docs/ARCHITECTURE.md</code> — arquitectura resumida	Arquitectura con diagramas, capas y flujos completos (03)
<code>docs/COMMANDS.md</code> — comandos CLI	Catálogo de comandos con parámetros y códigos de salida (05, 09)
<code>docs/HEURISTICAS.md</code> — heurísticas	Umbrales exactos, puntajes y <code>kind</code> verificados en el código (06)
<code>docs/DETECCION_AMENAZAS.md</code> — mapa amenaza→detección	Trazabilidad señal → función → persistencia → prueba (19)
<code>docs/PERSISTENCIA_MACOS.md</code> — persistencia en macOS	Implementación real del escaneo y su clasificación (06)
<code>docs/INDEX.md</code> — índice de docs de producto	Índice de documentación de sistema (este archivo)

9. Convenciones utilizadas

9.1 Marcadores de confianza

Toda afirmación de estos documentos está anclada a evidencia del repositorio. Cuando algo no se pudo comprobar, se marca explícitamente:

Marcador	Significado
(sin marcador)	Hecho verificado: leído directamente en el código, la configuración o el historial del repositorio. Se cita archivo y, cuando aplica, símbolo.
INFERENCIA	Conclusión razonada a partir del código, no una afirmación literal del repositorio.
REQUIERE VALIDACIÓN	Depende de ejecución en un Mac con permisos concedidos, de servicios externos o de una decisión humana pendiente.
NO DOCUMENTADO EN EL REPOSITORIO	El repositorio no contiene la información.
NO IDENTIFICADO	Se buscó y no se encontró evidencia en ningún sentido.

9.2 Referencias a código

- Los archivos se citan con ruta relativa a la raíz del repositorio: `src/services/inspector.rs`.
- Los símbolos se citan con su nombre real, sin traducir: `InspectorService::collect_snapshot`.
- Las líneas citadas corresponden al commit analizado y pueden desplazarse en commits posteriores; el nombre del símbolo es la referencia estable.

9.3 Idioma

Documentación en español, igual que el resto del repositorio y que los comentarios del código. Los identificadores de código, los `kind` de anomalía, los nombres de columnas SQL y los servicios TCC (`kTCCService...`) se mantienen en su forma original para garantizar trazabilidad textual.

9.4 Datos sensibles

Ningún documento contiene credenciales, tokens ni claves reales. Los valores de ejemplo son ficticios y están marcados como tales. Los hallazgos de seguridad se describen sin publicar valores explotables.

10. Elementos pendientes de validar

Lista viva de lo que esta documentación **no** pudo verificar en este análisis. El detalle y la recomendación de cada punto están en [15 · Riesgos y deuda técnica](#).

#	Pendiente	Motivo	Documento
1	Lectura real de <code>TCC.db</code>	Requiere conceder Acceso total al disco al binario o al <code>.app</code> ; sin ese permiso la sección declara la limitación en vez de mostrar datos	06 , 11
2	Comportamiento con privilegios de root	<code>lsof</code> sin root solo ve los sockets del propio usuario; el código lo declara, pero no se ejecutó como root en este análisis	08 , 11

3	Consulta de login items vía <code>osascript</code>	Dispara el diálogo de permiso de Automatización de macOS; no se ejecutó para no alterar el estado de permisos del equipo	09
4	Barrido activo de red (<code>network --deep</code>)	Genera tráfico ICMP hacia 254 direcciones del segmento; no se ejecutó en este análisis	06
5	Contrato exacto del proveedor IA configurado	El endpoint es configurable por el usuario; el repositorio solo asume forma compatible con la API de chat de OpenAI	09
6	Vulnerabilidades conocidas de las dependencias	No se ejecutó <code>cargo audit</code> ni consulta a bases de CVE en este análisis	11
7	Publicación efectiva del cask de Homebrew	<code>packaging/homebrew/rootcause.rb</code> es una plantilla; no hay tap publicado	13
8	Renderizado de los diagramas Mermaid en los PDF	El generador de PDF no ejecuta JavaScript: los diagramas viajan como código fuente legible, no como imagen	13

Documentación generada por análisis estático del repositorio en el commit `2282a33`, con ejecución local de `cargo fmt`, `cargo clippy`, `cargo test` y de los comandos de solo lectura del CLI. Si el código cambia, actualizar primero los Markdown de esta carpeta y regenerar los PDF.